几何计算前沿　第一次作业

TSDF Fusion 实现报告

刘智琦 2300012860

2026 年 4 月 4 日

1　概述

本次作业实现了 TSDF (Truncated Signed Distance Function) Fusion 算法。该算法是 KinectFusion 的简化版本，通过融合 1000 帧 RGB-D 图像构建三维场景的隐式表面表示，最终使用 Marching Cubes 算法提取三角网格。此外，作为附加任务，实现了带颜色信息的 TSDF Fusion。

2　输入数据

输入数据来自 7-scenes 数据集，包含：

- 1000 张深度图（16-bit PNG，单位毫米）
- 1000 张彩色图（JPEG）
- 1000 个相机位姿矩阵（4×4 刚体变换矩阵）
- 相机内参矩阵（$f_x = f_y = 585$，$c_x = 320$，$c_y = 240$）

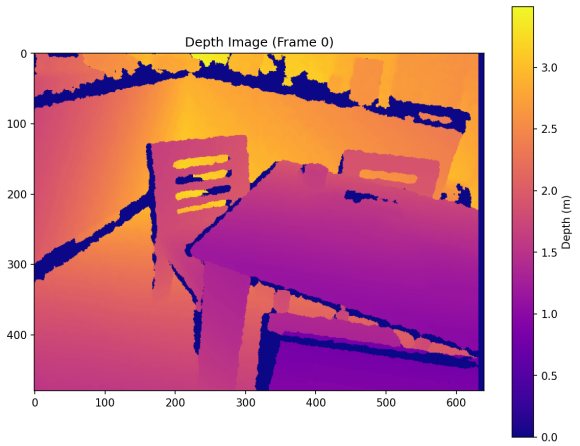


图 1: 第 0 帧深度图



图 2: 第 0 帧彩色图

3 Task 1: 从深度图生成点云

3.1 算法描述

给定深度图像素坐标 (u, v) 和对应深度值 z，首先利用相机内参将其转换到相机坐标系：

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = z \begin{bmatrix} 1/f_x & 0 & -c_x/f_x \\ 0 & 1/f_y & -c_y/f_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \tag{1}$$

即：

$$x = \frac{(u - c_x) \cdot z}{f_x}, \quad y = \frac{(v - c_y) \cdot z}{f_y} \tag{2}$$

然后利用相机位姿矩阵 T_{cw}（camera-to-world）将相机坐标转换为世界坐标：

$$\mathbf{p}_w = T_{cw} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \tag{3}$$

3.2 实现

```
def cam_to_world(depth_im, cam_intr, cam_pose, export_pc=False):
    H, W = depth_im.shape
    fx, fy = cam_intr[0, 0], cam_intr[1, 1]
    cx, cy = cam_intr[0, 2], cam_intr[1, 2]

    u, v = np.meshgrid(np.arange(W), np.arange(H))
```

```
    z = depth_im
    valid = z > 0
    u, v, z = u[valid], v[valid], z[valid]

    x = (u - cx) * z / fx
    y = (v - cy) * z / fy
    cam_pts = np.stack([x, y, z], axis=-1)

    cam_pts_h = np.concatenate(
        [cam_pts, np.ones((cam_pts.shape[0], 1))], axis=1)
    world_pts = (cam_pose @ cam_pts_h.T).T[:, :3]
    return world_pts
```

在 `demo.py` 中，遍历所有帧的点云，取各轴的最小/最大值确定体素场范围 `vol_bnds`。

3.3 结果

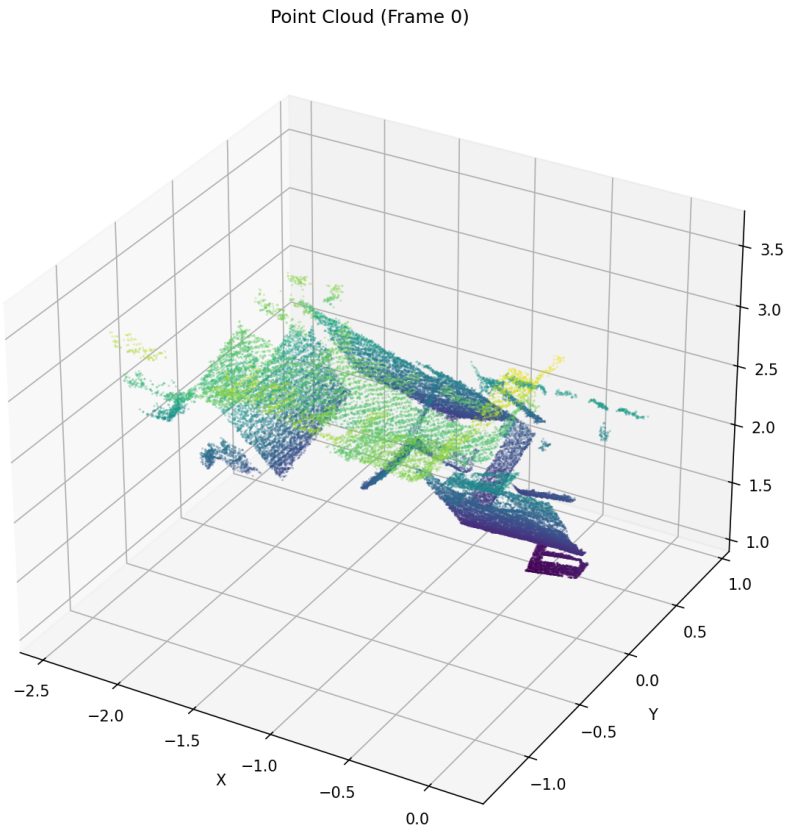


图 3: 第 0 帧深度图转换得到的点云（世界坐标系）

经过 1000 帧扫描，得到的体素场范围为：

- X: $[-2.97, 3.81]$ m
- Y: $[-1.91, 1.03]$ m
- Z: $[0.98, 3.91]$ m

4 Task 2: 从深度图采样

4.1 算法描述

本步骤将 TSDF 体素场的采样点投影到深度图上进行深度采样，是 Task 1 的逆过程。具体步骤：

1. **建立体素网格**：根据体素场范围 `vol_bnds` 和体素尺寸 `voxel_size`（2cm），计算各轴方向的体素数量 $\text{dim}_i = \lceil (\max_i - \min_i)/\text{voxel_size} \rceil$，建立整数坐标网格。
2. **体素坐标 → 世界坐标**：$\mathbf{p}_w = \text{origin} + \text{vox_coords} \times \text{voxel_size}$
3. **世界坐标 → 相机坐标**：$\mathbf{p}_c = T_{cw}^{-1}\mathbf{p}_w$
4. **相机坐标 → 像素坐标**：$u = f_x \cdot x/z + c_x$，$v = f_y \cdot y/z + c_y$
5. **深度采样**：对落在图片范围内且 $z > 0$ 的体素点，从深度图上采样对应深度值。

4.2 实现

初始化部分：

```
self._vol_dim = np.ceil(
    (vol_bnds[:, 1] - vol_bnds[:, 0]) / self.voxel_size
).astype(int)
self._vol_origin = vol_bnds[:, 0]
self.tsdf_vol = np.ones(self._vol_dim)
self.weight_vol = np.zeros(self._vol_dim)
self.color_vol = np.zeros((*self._vol_dim, 3))

xv, yv, zv = np.meshgrid(
    np.arange(self._vol_dim[0]),
    np.arange(self._vol_dim[1]),
    np.arange(self._vol_dim[2]),
    indexing='ij')
self.vox_coords = np.stack([xv, yv, zv], axis=-1).reshape(-1, 3)
```

投影与采样部分（在 integrate 方法中）：

```
# Voxel -> world -> camera coordinates
world_pts = self._vol_origin + self.vox_coords * self.voxel_size
world_pts_h = np.concatenate(
    [world_pts, np.ones((world_pts.shape[0], 1))], axis=1)
cam_pts = (np.linalg.inv(cam_pose) @ world_pts_h.T).T
cam_z = cam_pts[:, 2]

# Camera -> pixel coordinates
pix_x = np.round(fx * cam_pts[:, 0] / cam_z + cx).astype(int)
pix_y = np.round(fy * cam_pts[:, 1] / cam_z + cy).astype(int)

# Filter valid pixels and sample depth
valid = (pix_x >= 0) & (pix_x < im_w) \
      & (pix_y >= 0) & (pix_y < im_h) & (cam_z > 0)
depth_val = depth_im[pix_y[valid_idx], pix_x[valid_idx]]
```

5 Task 3: 计算单帧 TSDF

5.1 算法描述

对于每个有效的体素采样点，根据其在相机坐标系下的 z 值和深度图上采样得到的深度值 d，计算 TSDF 值：

$$\text{TSDF} = \min\left(1.0,\ \frac{d - z}{t}\right) \tag{4}$$

其中 t 为截断距离（truncation margin），设为 $5 \times \text{voxel_size} = 0.1$ m。

TSDF 的物理含义：

- TSDF > 0：该点在表面前方（相机侧）
- TSDF < 0：该点在表面后方
- TSDF $= 0$：该点恰好在表面上

只保留 TSDF ≥ -1.0 的体素点，丢弃距离表面过远的点。

5.2 实现

```
tsdf = np.minimum(1.0,
    (depth_val - cam_z[valid_idx]) / self.trunc_margin)
valid_tsdf = tsdf >= -1.0
valid_idx = valid_idx[valid_tsdf]
tsdf = tsdf[valid_tsdf]
```

6 Task 4: 融合多帧 TSDF 并提取 Mesh

6.1 算法描述

将多帧 TSDF 通过加权平均方式融合。设第 i 帧的 TSDF 值为 $d_i(x)$，对应权重为 w_i，则融合公式为：

$$D_{i+1}(x) = \frac{W_i(x) \cdot D_i(x) + w_{i+1} \cdot d_{i+1}(x)}{W_i(x) + w_{i+1}} \tag{5}$$

$$W_{i+1}(x) = W_i(x) + w_{i+1} \tag{6}$$

所有帧融合完成后，使用 Marching Cubes 算法在 TSDF 场的零等值面处提取三角网格。Marching Cubes 通过遍历每个体素，根据 8 个顶点的 TSDF 正负号确定等值面与体素边的交点，进而构建三角面片。这里直接调用 `skimage.measure.marching_cubes` 实现。

6.2 实现

融合部分：

```
w_old = self.weight_vol[vx, vy, vz]
d_old = self.tsdf_vol[vx, vy, vz]
w_new = w_old + obs_weight
self.tsdf_vol[vx, vy, vz] = \
    (w_old * d_old + obs_weight * tsdf) / w_new
self.weight_vol[vx, vy, vz] = w_new
```

Mesh 提取部分（在 `demo.py` 中）：

```
verts, faces, normals, _ = measure.marching_cubes(
    tsdf_vol.tsdf_vol, level=0)
verts = verts * tsdf_vol.voxel_size + tsdf_vol._vol_origin
mesh = trimesh.Trimesh(vertices=verts, faces=faces,
                       vertex_normals=normals)
mesh.export("mesh.ply")
```

6.3 结果

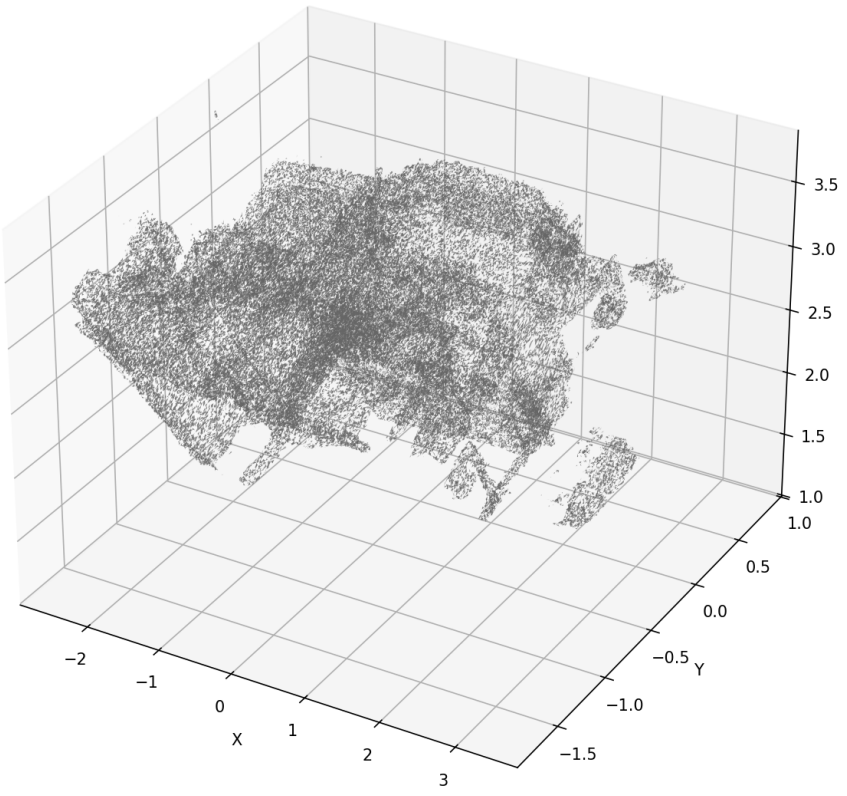


图 4: TSDF Fusion 后通过 Marching Cubes 提取的 Mesh（无颜色）

融合 1000 帧的平均处理速度为 1.35 FPS，体素分辨率为 2cm。

7 附加任务: 带颜色的 TSDF Fusion

7.1 算法描述

在 TSDF 融合的同时，对颜色信息也进行加权平均融合。对于每个有效体素点，从当前帧的彩色图上采样颜色值（使用与深度采样相同的像素坐标），然后使用与 TSDF 融合相同的权重进行加权平均:

$$C_{i+1}(x) = \frac{W_i(x) \cdot C_i(x) + w_{i+1} \cdot c_{i+1}(x)}{W_i(x) + w_{i+1}} \tag{7}$$

提取 Mesh 后，将颜色体积中对应顶点位置的颜色值赋给 Mesh 的顶点颜色。

7.2 实现

颜色融合（在 `integrate` 中）:

```
if color_im is not None:
    px = pix_x[valid_idx]
    py = pix_y[valid_idx]
    new_color = color_im[py, px, :]
    c_old = self.color_vol[vx, vy, vz]
    self.color_vol[vx, vy, vz] = \
        (w_old[:, None] * c_old + obs_weight * new_color) \
        / w_new[:, None]
```

顶点着色（在 demo.py 中）：

```
verts_idx = np.round(
    (verts - tsdf_vol._vol_origin) / tsdf_vol.voxel_size
).astype(int)
verts_idx = np.clip(verts_idx, 0,
                    np.array(tsdf_vol._vol_dim) - 1)
vert_colors = tsdf_vol.color_vol[
    verts_idx[:, 0], verts_idx[:, 1], verts_idx[:, 2]]
vert_colors = vert_colors[:, ::-1].astype(np.uint8)
```

7.3 结果

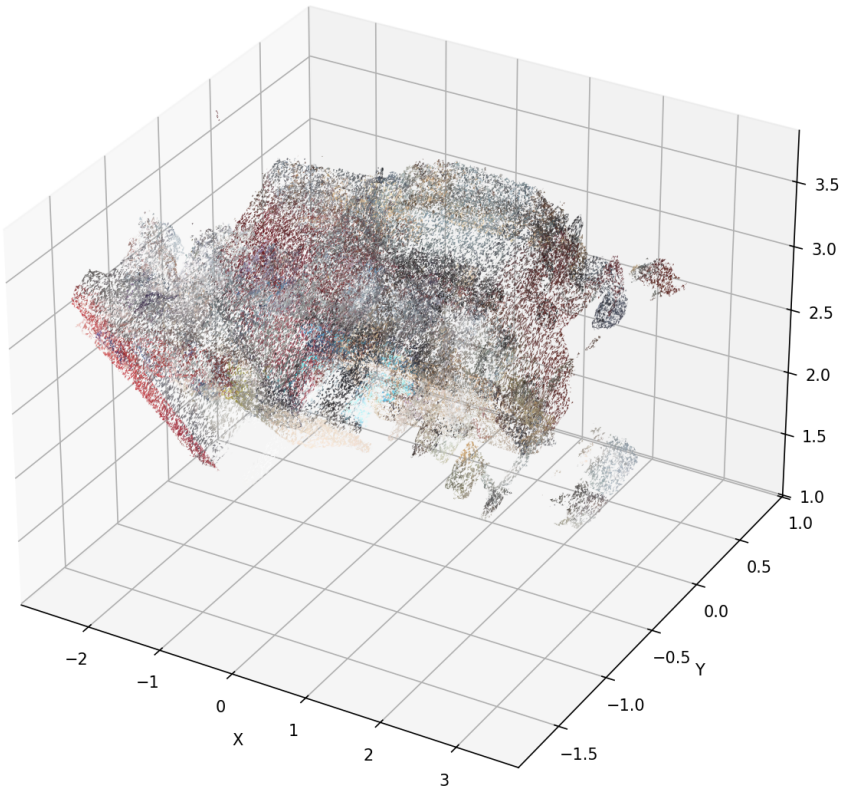


图 5: 带颜色的 TSDF Fusion Mesh

可以看到，带颜色的 Mesh 保留了场景中物体的纹理信息，包括桌面的木纹、红色橱柜、地面的灰色条纹等细节。

8 总结

本次作业完成了 TSDF Fusion 的完整流程：

1. 将深度图转换为世界坐标系下的点云，确定体素场范围
2. 建立体素网格并将体素点投影到深度图进行深度采样
3. 计算单帧的截断符号距离函数值
4. 通过加权平均融合多帧 TSDF，并使用 Marching Cubes 提取三角网格
5. 实现了带颜色信息的 TSDF Fusion（附加任务）

最终输出的 Mesh 文件（`mesh.ply` 和 `mesh_no_color.ply`）可以使用 MeshLab 等工具进行可视化。